Beginner Guide to AKS Cluster Autoscaler Setup

The AKS Cluster Autoscaler automatically adds or removes VM nodes in your Kubernetes cluster based on workload demand. Learning this helps you prevent app downtime during traffic surges while saving cloud costs when traffic is low.

What you'll learn:

1. Enable Autoscaler on New Cluster
2. Enable Autoscaler on Existing Pool
3. Check Autoscaler Health and Status
4. Adjust Min and Max Boundaries

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Enable Autoscaler on New Cluster

When creating a new Azure Kubernetes Service cluster, you can enable the cluster autoscaler right from the start. You define a minimum node count so your application always has baseline resources, and a maximum node count to prevent run-away cloud spending. Kubernetes will automatically request extra Azure VMs whenever pods cannot be scheduled due to memory or CPU limits.

```
$az aks create \
   --resource-group myResourceGroup \
   --name myAKSCluster \
   --node-count 2 \
   --enable-cluster-autoscaler \
   --min-count 2 \
   --max-count 6
```

AKS Cluster Autoscaler: Scale Nodes Up & Down Automatically

Thursday, November 12, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Enable Autoscaler on Existing Pool

If you already have a running cluster without autoscaling, you do not need to rebuild it. You can update your existing node pool in-place to enable the autoscaler. Once enabled, the control plane checks for unschedulable pods every few seconds and triggers Azure to provision new worker nodes automatically.

```
$az aks update \
   --resource-group myResourceGroup \
   --name myAKSCluster \
   --enable-cluster-autoscaler \
   --min-count 1 \
   --max-count 8
```

AKS Cluster Autoscaler: Scale Nodes Up & Down Automatically

Thursday, November 12, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Check Autoscaler Health and Status

To understand why the autoscaler is scaling up, holding back, or scaling down, you can inspect its status configmap. This resource is maintained by Kubernetes inside the kube-system namespace. It gives you direct visibility into node group health, unneeded nodes, and unschedulable pod counts.

```
$kubectl describe configmap cluster-autoscaler-status -n kube-system
```

AKS Cluster Autoscaler: Scale Nodes Up & Down Automatically

Thursday, November 12, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Adjust Min and Max Boundaries

As your application traffic grows, your initial scaling limits might become too small. You can update the autoscaler boundaries live without causing cluster downtime. Increasing the max-count gives your cluster room to grow during heavy traffic events like product launches.

```
$az aks update \
   --resource-group myResourceGroup \
   --name myAKSCluster \
   --update-cluster-autoscaler \
   --min-count 3 \
   --max-count 12
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Set realistic resource requests (CPU/Memory) on your pods so the autoscaler knows when nodes are full.
+ Always configure pod disruption budgets to prevent critical apps from being killed during scale-down.
+ Set sensible max-count values on node pools to avoid unexpected Azure bill surges.
+ Combine Cluster Autoscaler with Horizontal Pod Autoscaler (HPA) for complete auto-scaling.
+ Monitor cluster-autoscaler logs in Azure Log Analytics to troubleshoot slow node scale-ups.

Watch Out For

! Confusing Pod scaling (HPA) with Node scaling (Cluster Autoscaler).
! Forgetting to set CPU/Memory requests on containers, causing the autoscaler to never trigger.
! Setting min-count too high, which wastes money on idle Virtual Machines.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh